

Structure-Gated Mode-A CDCL Branching

Instance-Level Triage of Structural Search in MAAT v1.6-T

Christof Krieg

MAAT Project – Independent Research

<https://maat-research.com>

2026

Abstract

Paper 62 implemented the first policy-level test of MAAT v1.6-T: Mode-A structural acquisition inside a transparent CDCL solver. The result was mixed. Global $h = 0$ Mode A tied VSIDS in median compute cost but had positive mean regret and did not beat the progress-only ablation. Family-level analysis showed the real signal: Mode A helped in some structure-rich families and failed sharply in others.

This paper tests the next hypothesis: structural search should be *gated* by an instance-level structure diagnostic. Instead of applying the same structural penalty everywhere, we use

$$A_g(c) = \text{prog}(c) - g(\mathcal{F})\tau h_{\text{MAAT}}(c),$$

where $g(\mathcal{F}) \in [0, 1]$ is computed from root-level clause-variable graph diagnostics without using family labels. The gate combines co-occurrence clustering, V -mode spread, degree-tail pressure, and a symmetry penalty.

The result is again mixed but informative. Gated Mode A reduces mean regret relative to global Mode A (-0.5703 paired mean regret, median -0.008 , 50 wins and 45 losses), and correctly suppresses the structural penalty on pigeonhole formulas ($g = 0$). However, it still does not beat progress-only globally and assigns overly high gate values to random and XOR instances. Thus Paper 63 supports the core v1.6-T lesson: structure-gating can reduce tail damage, but the present hand-built gate is not yet selective enough to turn Mode A into a robust CDCL improvement.

1 Purpose

MAAT v1.6-T defines structural search as a pre-inference triage layer. Paper 62 instantiated its first obligation, T1, by inserting Mode A into a transparent CDCL solver:

$$A(c) = \text{prog}(c) - \tau h_{\text{MAAT}}(c).$$

The test did not produce a global win. Instead, it produced a sharper diagnosis:

structural steering helps where exploitable structure is present and hurts where structure is absent, symmetric, or misleading.

The natural next question is therefore not whether to increase or decrease τ globally. It is whether the solver can first decide when structural steering should be active.

2 Structure-Gated Mode A

The tested policy is

$$\boxed{A_g(c) = \text{prog}(c) - g(\mathcal{F})\tau h_{\text{MAAT}}(c)} \tag{1}$$

where $\mathcal{F}$ is the CNF instance and $g(\mathcal{F}) \in [0, 1]$ is an instance-level gate. When $g = 0$, the policy collapses to progress-only. When $g = 1$, it recovers global Mode A.

The gate is computed once at the root level from the variable co-occurrence graph. It does not use family labels. The diagnostic components are:

- mean co-occurrence clustering,
- standard deviation of local connectivity support V ,
- high-degree tail pressure,
- a symmetry penalty suppressing nearly uniform graphs.

In schematic form:

$$g(\mathcal{F}) = \text{clip}_{[0,1]} [0.52 C_{\text{comm}} + 0.30 V_{\text{spread}} + 0.18 T_{\text{deg}} - 0.55 P_{\text{sym}}] \quad (2)$$

This is a deliberately simple hand-built gate. It is not trained and not claimed to be optimal. Its purpose is to test the qualitative v1.6-T hypothesis:

triage should be applied to the triage layer itself.

3 Benchmark Design

The solver and instance families are inherited from Paper 62. The CDCL implementation is transparent and small: unit propagation, conflict detection, learned clauses, VSIDS bumping and decay, phase memory, and a fixed conflict budget. There are no restarts, watched literals, LRB/CHB, clause database reduction, or industrial SAT-solver optimizations.

The fixed settings are:

Parameter	Value
seed	63063
conflict budget	3500
β	8.0
τ	0.38
rollout horizon h	0
instances	99

The tested policies are:

VSIDS, progress-only, MAAT-only, global Mode A, gated Mode A, random.

The compute cost is the same as in Paper 62:

$$C_{\text{compute}} = C + 0.30D + 0.004P + TB_{\text{conf}},$$

where C is conflicts, D decisions, P propagations, and T a timeout indicator.

4 Aggregate Results

All policies solve all 99 instances. The benchmark therefore measures compute ordering, not timeout separation.

Policy	Solved	Med. conflicts	Med. decisions	Med. cost	Mean cost
Mode A	1.000	4.0	7.0	5.252	20.5568
Progress-only	1.000	4.0	7.0	5.260	13.2762
Gated Mode A	1.000	4.0	8.0	5.268	19.9865
VSIDS	1.000	4.0	7.0	5.584	12.8841
MAAT-only	1.000	5.0	9.0	7.092	24.2539
Random	1.000	4.0	10.0	7.212	27.5876

Table 1: Aggregate policy statistics. Gated Mode A slightly lowers mean cost relative to global Mode A, but it still does not beat progress-only or VSIDS in mean cost.

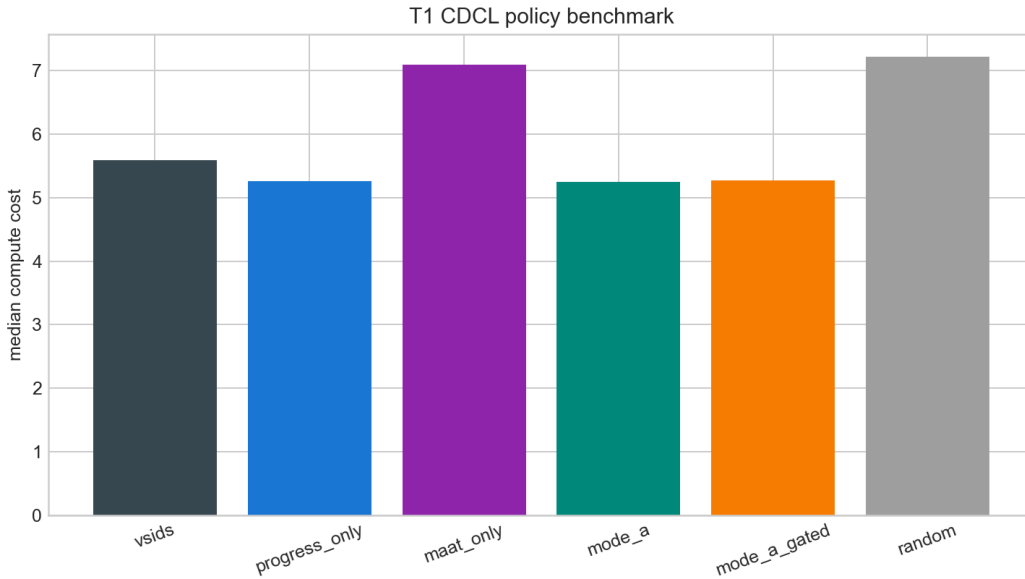


Figure 1: Median compute cost by policy. Gated Mode A is competitive in median cost but is not a global improvement over progress-only or VSIDS.

5 Regret against VSIDS

Policy	Mean regret	Med. regret	Mean ratio	Wins	Losses
Progress-only	+0.3921	0.000	0.9781	32	24
MAAT-only	+11.3698	+1.332	1.4904	35	63
Mode A	+7.6727	0.000	1.0726	47	44
Gated Mode A	+7.1023	0.000	1.1627	47	48
Random	+14.7035	+1.332	1.7462	31	63

Table 2: Paired regret against VSIDS. Negative regret would mean lower compute than VSIDS. Gated Mode A reduces mean regret relative to global Mode A but does not become globally competitive.



Figure 2: Median regret against VSIDS. Gated Mode A remains neutral in the median and positive in the mean.

6 Does Gating Improve Mode A?

The more direct question is whether the gate improves global Mode A. Against global Mode A, gated Mode A has:

$$\Delta C_{\text{gated-modeA}} = -0.5703 \quad (\text{mean}), \quad -0.008 \quad (\text{median}).$$

It records 50 wins, 45 losses, and 4 ties.

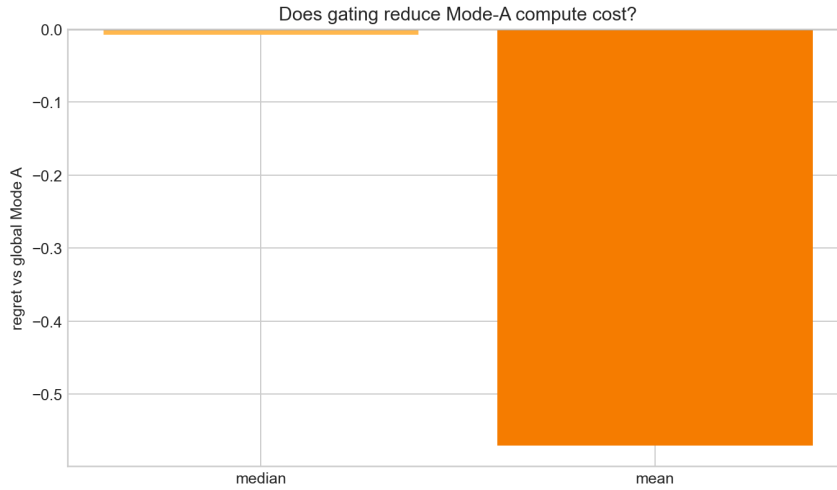


Figure 3: Paired regret of gated Mode A against global Mode A. Negative values mean the gate reduced compute relative to the ungated structural policy.

This is a small but real positive result. The gate reduces global Mode-A damage. However, the improvement is not large enough to beat progress-only globally. Thus the gate is a partial correction, not a full solution.

7 Family-Level Results

Family	Progress-only	Mode A	Gated Mode A	Gate
graph coloring	0.000	+0.148	-0.296	0.522
modular 3-SAT	0.000	+0.286	+0.016	0.590
pigeonhole	+14.904	+56.520	+42.532	0.000
planted 3-SAT	-0.296	-0.300	-0.120	0.563
random 3-SAT	0.000	+0.292	+1.192	0.581
XOR CNF	0.000	0.000	-0.008	0.749

Table 3: Family-level median regret against VSIDS and median gate value. Negative regret means lower compute cost than VSIDS. The gate successfully suppresses pigeonhole but remains too permissive on random and XOR families.

The gate column should be read diagnostically. The desired behavior is not simply “large gate is good”. A useful gate should activate on exploitable low-frequency structure and suppress itself on symmetry-dominated or locally misleading structure. In this benchmark, pigeonhole is correctly suppressed, while random and XOR-like formulas remain overactivated.

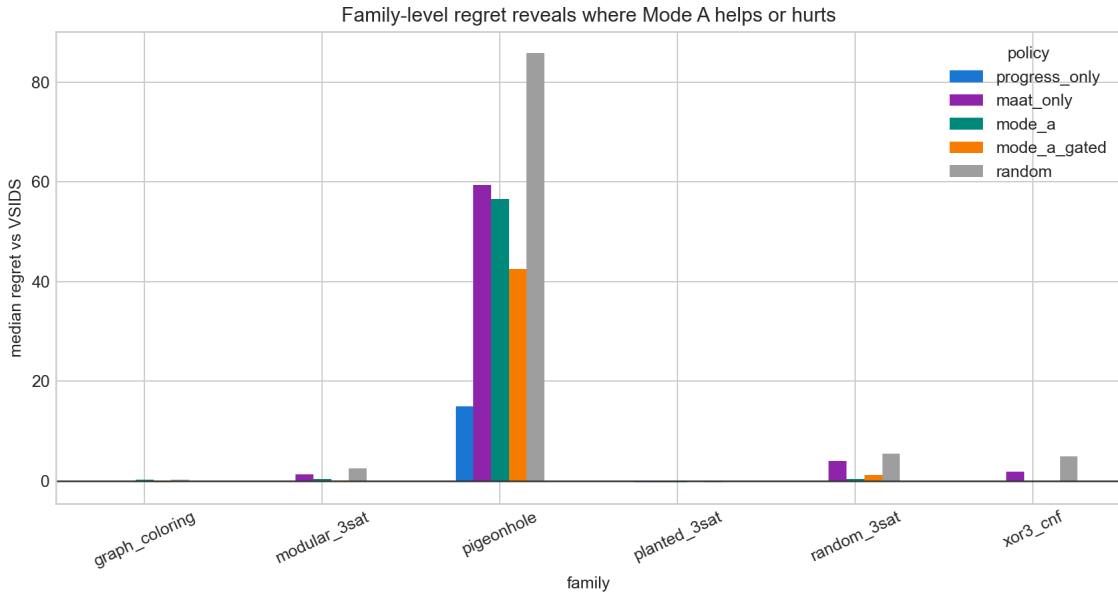


Figure 4: Family-level regret against VSIDS. Gating improves some structured families and reduces pigeonhole damage, but does not solve random-family overactivation.

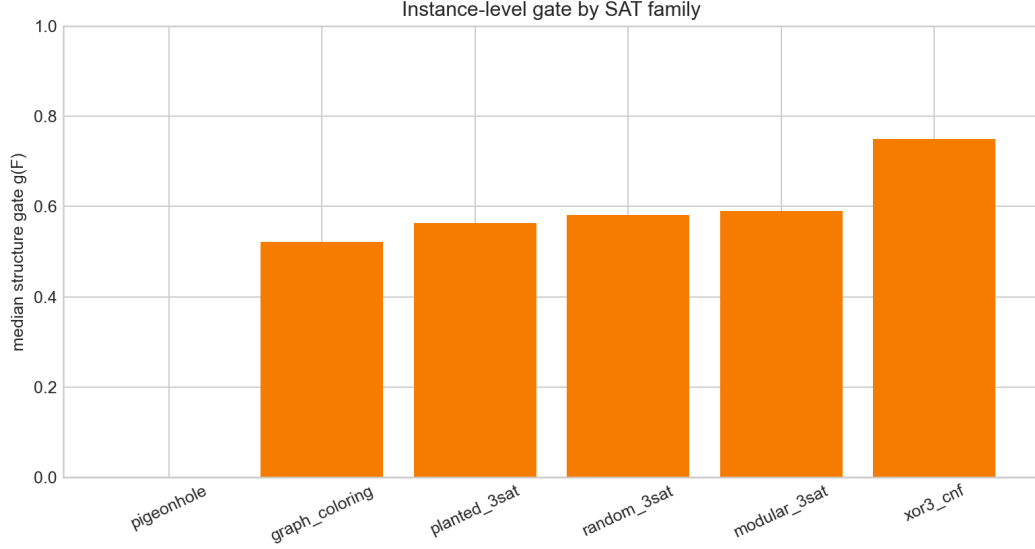


Figure 5: Median structure gate by family. The correct suppression of pigeonhole is the main success; the high gate for random and XOR instances is the main failure mode.

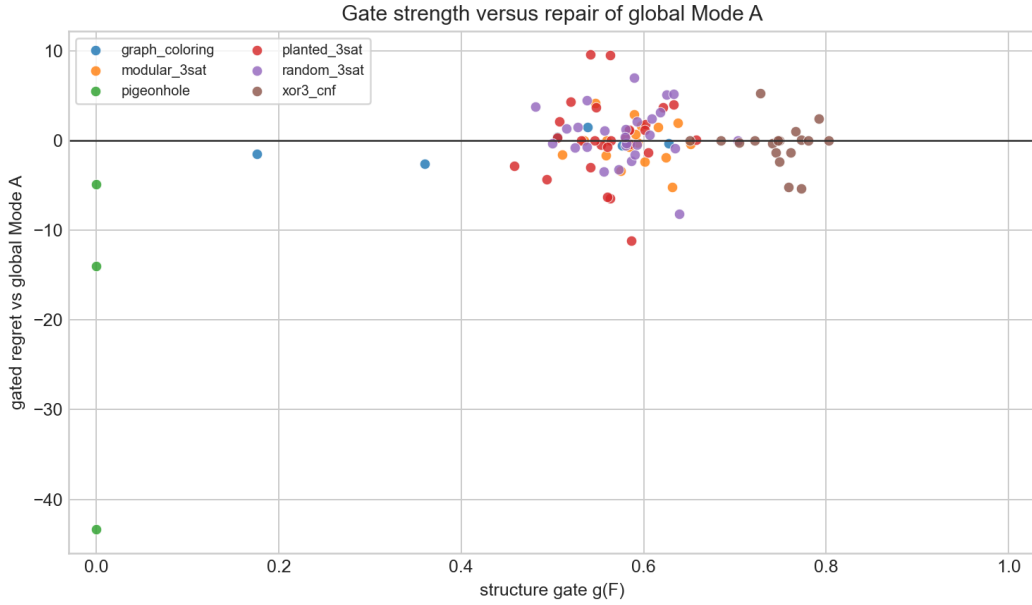


Figure 6: Instance-level gate value versus paired regret of gated Mode A against global Mode A. Negative values indicate that gating repaired the ungated structural policy on that instance. The scatter shows why the current gate is useful but too coarse: it suppresses one damaging family but does not yet cleanly separate helpful from harmful structural activation.

8 Interpretation

Paper 63 answers the direct question:

Can a simple instance-level gate repair global $h = 0$ Mode A?

The answer is:

partially, but not enough.

The gate performs the intended role in one important respect: it suppresses the structural penalty on pigeonhole formulas, where global Mode A had large regret. It also improves graph coloring and slightly improves XOR CNF. But it overactivates on random 3-SAT and does not preserve the stronger performance of global Mode A on planted formulas.

This result is useful because it narrows the v1.6-T programme. The next gate cannot be merely a generic graph-structure detector. It must distinguish:

- exploitable low-frequency structure,
- misleading random local clustering,
- symmetry-dominated global constraints,
- structural features already captured by progress-only CDCL signals.

9 Limitations

- The solver is still a transparent pilot CDCL solver, not an industrial SAT solver.
- All instances are synthetic and all policies solve all instances. The test measures compute ordering, not hard timeout separation.
- The gate is hand-built and not optimized. It is deliberately label-free, but it is also too coarse.
- The label-free design is a strength for generality, but it also explains why the gate cannot yet exploit family-specific structural patterns that a learned or calibrated gate might capture.
- The benchmark tests only $(\beta, \tau, h) = (8.0, 0.38, 0)$.
- LRB, CHB, watched literals, restarts, clause database reduction, and SAT Competition instances remain future work.

10 Reproducibility

Repository:

<https://github.com/Chris4081/structural-selection-principle>

Experiment folder:

`experiments/sat_cdcl_structure_gated_mode_a_paper63/`

Run:

```
cd experiments/sat_cdcl_structure_gated_mode_a_paper63
python3 sat_cdcl_structure_gated_mode_a_paper63.py
```

Outputs include:

- `paper63_cdcl_runs.csv`,
- `paper63_structure_gates.csv`,
- `paper63_summary_by_policy.csv`,
- `paper63_regret_vs_vsids.csv`,

- `paper63_regret_vs_mode_a.csv`,
- `paper63_family_regret_vs_vsids.csv`,
- `paper63_summary.json`,
- six PNG figures.

Data attribution and license note. No external CNF dataset is redistributed. Instances are generated locally from standard synthetic SAT-family generators. CSV, JSON, and PNG files are derived reproducibility artifacts. Repository code is distributed under the MIT License. No endorsement by the authors of DPLL, CDCL, VSIDS, or any SAT-solver project is implied.

11 Conclusion

Paper 63 tests the first repair suggested by Paper 62: do not apply structural search pressure everywhere. Gate it.

The result is a partial success. Gated Mode A reduces mean regret relative to global Mode A and correctly turns off structural steering on pigeonhole formulas. But it remains worse than progress-only globally and still overactivates on random-like formulas.

The main contribution is therefore diagnostic: structure-gated triage is the right direction, but the current gate is too coarse. The next MAAT v1.6-T step should test learned or predeclared multi-feature gates against stronger CDCL baselines and harder external SAT instances.

References

- [1] M. Davis, G. Logemann, and D. Loveland, “A machine program for theorem-proving,” *Communications of the ACM* **5**, 394–397 (1962).
- [2] J. P. Marques-Silva and K. A. Sakallah, “GRASP: A search algorithm for propositional satisfiability,” *IEEE Transactions on Computers* **48**, 506–521 (1999).
- [3] M. W. Moskewicz, C. F. Madigan, Y. Zhao, L. Zhang, and S. Malik, “Chaff: Engineering an efficient SAT solver,” in *Proceedings of the 38th Design Automation Conference*, 530–535 (2001).
- [4] A. Biere, M. Heule, H. van Maaren, and T. Walsh (eds.), *Handbook of Satisfiability*, IOS Press, Amsterdam (2009).
- [5] C. Krieg, “MAAT v1.6-T: The Triage Instantiation,” framework specification (2026).
- [6] C. Krieg, “Mode-A CDCL Branching,” preprint (2026).